

TOPIC 05

ELEMENTARY PROGRAMMING

(PART 1)

Fundamentals of Computing
(FSPK0022)

What Is a Program Made Of?

- Common elements in programming languages:
 - Key Words
 - Programmer-Defined Identifiers
 - Operators
 - Punctuation
 - Syntax

Key Words

- Also known as **reserved words**
- Have a special meaning in C++
- Can not be used for another purpose
- Written using lowercase letters
- Examples in program (shown in green):

```
using namespace std;  
int main()
```

Example Program

```
#include <iostream>
using namespace std;

int main()
{
    double num1 = 5,
           num2, sum;
    num2 = 12;
    sum = num1 + num2;
    cout << "The sum is " << sum;
    return 0;
}
```

Operators

- Used to perform operations on data
- Many types of operators
 - Arithmetic: $+$, $-$, $*$, $/$
 - Assignment: $=$
- Examples in program (shown in green):
`num2 = 12;`
`sum = num1 + num2;`

Example Program

```
#include <iostream>
using namespace std;

int main()
{
    double num1 = 5, num2, sum;
    num2 = 12;

    sum = num1 + num2;
    cout << "The sum is " << sum;
    return 0;
}
```

Punctuation

- Characters that mark the end of a statement, or that separate items in a list
- Example in program (shown in green):

```
double num1 = 5,  
           num2, sum;  
num2 = 12;
```

Example Program

```
#include <iostream>
using namespace std;

int main()
{
    double num1 = 5,
           num2, sum;
    num2 = 12;

    sum = num1 + num2;
    cout << "The sum is " << sum;
    return 0;
}
```


The `#include` Directive

- Inserts the contents of another file into the program
- Is a preprocessor directive
 - Not part of the C++ language
 - Not seen by compiler
- Example:

```
#include <iostream>
```



No ; goes here

Comments

- Are used to document parts of a program
- Are written for persons reading the source code of the program
 - Indicate the purpose of the program
 - Describe the use of variables
 - Explain complex sections of code
- Are ignored by the compiler

Single-Line Comments

- Begin with `//` through to the end of line

```
int length = 12; // length in inches
int width = 15;  // width in inches
int area;        // calculated area

// Calculate rectangle area
area = length * width;
```

Multi-Line Comments

- Begin with `/*` and end with `*/`
- Can span multiple lines

```
/*-----  
    Here's a multi-line comment  
-----*/
```

- Can also be used as single-line comments
`int area; /* Calculated area */`

The Parts of a C++ Program

Statement	Purpose
<code>//Sample C++ program</code>	comment
<code>#include <iostream></code>	preprocessor directive
<code>using namespace std;</code>	which namespace to use
<code>int main()</code>	beginning of function named <code>main</code>
<code>{</code>	beginning of block for <code>main</code>
<code> cout << "Hello, there!";</code>	output statement
<code> return 0;</code>	send 0 back to the operating system
<code>}</code>	end of block for <code>main</code>

Special Characters

Character	Name	Description
//	Double Slash	Begins a comment
#	Pound Sign	Begins preprocessor directive
< >	Open, Close Brackets	Encloses filename used in #include directive
()	Open, Close Parentheses	Used when naming function
{ }	Open, Close Braces	Encloses a group of statements
" "	Open, Close Quote Marks	Encloses string of characters
;	Semicolon	Ends a programming statement

Important Details

- C++ is case-sensitive. Uppercase and lowercase characters are different characters. 'Main' is not the same as 'main'.
- Every { must have a corresponding }, and vice-versa.

Variables

Variables

- A variable is a named location in computer memory (in RAM)
- It holds a piece of data
- It must be *defined* before it can be used
- Example variable definition:

```
double num1;
```

Example Program

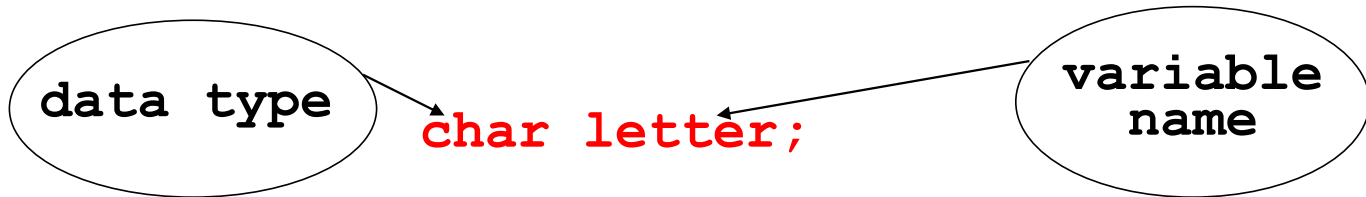
```
#include <iostream>
using namespace std;

int main()
{
    double num1 = 5,
           num2, sum;
    num2 = 12;

    sum = num1 + num2;
    cout << "The sum is " << sum;
    return 0;
}
```

Variables, Constants, and the Assignment Statement

- Variable
 - Has a name and a type of data it can hold



- Is used to reference a location in memory where a value can be stored
- Must be defined before it can be used
- The value that is stored can be changed, *i.e.*, it can “vary”

Variables

- If a new value is stored in the variable, it replaces the previous value
- The previous value is overwritten and can no longer be retrieved

```
int age;  
age = 17;      // age is 17  
cout << age;   // Displays 17  
age = 18;      // Now age is 18  
cout << age;   // Displays 18
```

Variables: Example

Program 2-7

```
1  // This program has a variable.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      int number;
8
9      number = 5;
10     cout << "The value in number is " << number << endl;
11     return 0;
12 }
```

Variable Definition



Program Output

The value in number is 5

Identifiers

Identifiers

- Programmer-chosen names to represent parts of the program, such as variables
- Name should indicate the use of the identifier
- Cannot use C++ key words as identifiers
- Must begin with alphabetic character or `_`, followed by alphabetic, numeric, or `_`. Alpha may be uppercase or lowercase
- Example in program (shown in green):

```
double num1;
```

Example Program

```
#include <iostream>
using namespace std;

int main()
{
    double num1 = 5,
           num2, sum;
    num2 = 12;

    sum = num1 + num2;
    cout << "The sum is " << sum;
    return 0;
}
```


Valid and Invalid Identifiers

Identifier	Valid?	Reason if Invalid
<code>totalSales</code>		
<code>total_Sales</code>		
<code>total.Sales</code>		
<code>4thQtrSales</code>		
<code>totalSale\$</code>		

Lines vs. Statements

In a source file,

A **line** is all of the characters entered before a carriage return.

Blank lines improve the readability of a program.

Here are four sample lines. Line 3 is blank:

```
double num1 = 5, num2, sum;  
num2 = 12;  
  
sum = num1 + num2;
```

Lines vs. Statements

In a source file,

A **statement** is an instruction to the computer to perform an action.

A statement may contain keywords, operators, programmer-defined identifiers, and punctuation.

A statement may fit on one line, or it may occupy multiple lines.

Here is a single statement that uses two lines:

```
double num1 = 5,  
       num2, sum;
```

Literals

- Literal: a value that is written into a program's code.
 - "hello, there" (string literal)
 - 12 (integer literal)

Literals: Example

Program 2-9

```
1 // This program has literals and a variable.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int apples;
8
9     apples = 20;
10    cout << "Today we sold " << apples << " bushels of apples.\n";
11    return 0;
12 }
```

20 is an integer literal



Program Output

Today we sold 20 bushels of apples.

Literals: Example

Program 2-9

```
1 // This program has literals and a variable.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int apples;
8
9     apples = 20;
10    cout << "Today we sold " << apples << " bushels of apples.\n";
11    return 0;
12 }
```

This is a string literal

Program Output

Today we sold 20 bushels of apples.

In-Class Exercise

Examine the following program. List all the variables and literals that appear in the program.

```
#include <iostream>
using namespace std;

int main()
{   int little;
    int big;

    little = 2;
    big = 2000;
    cout<<"The little number is " <<little<<endl;
    cout<<"The big number is " <<big<<endl;
    return 0;
}
```

In-Class Exercise

What will the following program display on the screen?

```
#include <iostream>
using namespace std;

int main()
{
    int num;
    num = 712;
    cout<< "The value is " << num << endl;
    return 0;
}
```


Data type and constant

Number Systems

- Numbers can be represented in a variety of ways.
- The representation depends on what is called the BASE.
- You write these numbers as:
 - **Number**_{base}

Number Systems

- The following are the four most common representations.
- Decimal (base 10)
 - Commonly used
 - Valid digits are from 0 to 9
 - Example: 126_{10} (normally written as just 126)
- Binary (base 2)
 - Valid digits are 0 and 1
 - Example: 1111110_2

- The following are the four most common representations.
- Octal (base 8)
 - Valid digits are from 0 to 7
 - Example: 176_8
- Hexadecimal (base 16)
 - Valid digits are from 0 to 9 and A to F (or from a to f)
 - Example: $7E_{16}$

Integer Data Types

- Designed to hold whole numbers
- Can be **signed** or **unsigned**

12 -6 +3

- Available in different sizes (*i.e.*, number of bytes): **short**, **int**, and **long**
- Size of **short** $\leq$ size of **int** $\leq$ size of **long**

Integral Constants

- To store an integer constant in a long memory location, put 'L' at the end of the number:

1234L

- Constants that begin with '0' (zero) are octal, or base 8: **075**
- Constants that begin with '0x' are hexadecimal, or base 16: **0x75A**

Defining Variables

- Variables of the same type can be defined

- In separate statements

```
int length;
```

```
int width;
```

- In the same statement

```
int length,
```

```
width;
```

- Variables of different types must be defined in separate statements

Floating-Point Data Types

- Designed to hold real numbers

12.45

-3.8

- Stored in a form similar to scientific notation
- Numbers are all signed
- 3 data types to represent floating-point numbers: **float**, **double**, and **long double**
- Size of **float** $\leq$ size of **double**
 $\leq$ size of **long double**

Floating-point Constants

- Can be represented in
 - Fixed point (decimal) notation:
31.4159 **0.0000625**
 - E notation:
3.14159E1 **6.25e-5**
- Are **double** by default
- Can be forced to be float **3.14159F** or long double **0.0000625L**

Assigning Floating-point Values to Integer Variables

If a floating-point value is assigned to an integer variable

- The fractional part will be truncated (*i.e.*, “chopped off” and discarded)
- The value is not rounded

```
int rainfall = 3.88;  
cout << rainfall; //Displays 3
```

The `bool` Data Type

- Represents values that are `true` or `false`
- `bool` values are stored as short integers
- `false` is represented by 0, `true` by 1

```
bool allDone = true;    allDone    finished
bool finished = false;  

|   |
|---|
| 1 |
|---|



|   |
|---|
| 0 |
|---|


```

The `char` Data Type

- Used to hold single characters or very small integer values
- Usually occupies 1 byte of memory
- A numeric code representing the character is stored in memory

SOURCE CODE

MEMORY

```
char letter = 'C'; letter
```

67

In-Class Exercise

- What is wrong with the following program?

```
#include <iostream>
using namespace std;

int main()
{
    char letter;

    letter = "Z";
    cout<<letter<<endl;
    return 0;

5 }
```

Summary of data types

Name	Description	Size	Range
char	Character or small integer.	1byte	signed: -128 to 127 unsigned: 0 to 255
short int (short)	Short Integer.	2bytes	signed: -32768 to 32767 unsigned: 0 to 65535
int	Integer.	4bytes	signed: -2147483648 to 2147483647 unsigned: 0 to 4294967295
long int (long)	Long integer.	4bytes	signed: -2147483648 to 2147483647 unsigned: 0 to 4294967295
bool	Boolean value. It can take one of two values: true or false.	1byte	true or false
float	Floating point number.	4bytes	+/- 3.4e +/- 38 (~7 digits)
double	Double precision floating point number.	8bytes	+/- 1.7e +/- 308 (~15 digits)
long double	Long double precision floating point number.	8bytes	+/- 1.7e +/- 308 (~15 digits)

Naming Constant

Named Constants

- Named constant (constant variable): variable whose content cannot be changed during program execution
- Used for representing constant values with descriptive names:

```
const double TAX_RATE = 0.0675;  
const int NUM_STATES = 50;
```

- Often named in uppercase letters

Defining constants (**#define**)

- You can define your own names for constants that you use very often without having to resort to memory-consuming variables, simply by using the **#define** preprocessor directive.

- Its format:

#define identifier value

- Example:

```
#include <iostream>
using namespace std;
#define PI 3.14159
```

```
#define NEWLINE '\n'
```

(Before int main())

```
int main ()
{ double r=5.0;
  double circle;
  circle = 2 * PI * r;
  cout << circle << NEWLINE;
  return 0; }
```

Declared constants (**const**)

- With the **const** prefix you can declare constants with a specific type in the same way as you would do with a variable
- Example:

```
#include <iostream>
using namespace std;
int main ()
{ double r=5.0, circle;
  const double PI = 3.14159;
  const char NEWLINE = '\\n';
  circle = 2 * PI * r;
  cout << circle << NEWLINE;
  return 0; }
```

(After int main ())

String Constant

- Can be stored a series of characters in consecutive memory locations

"Hello"

- Stored with the **null terminator**, `\0`, at end

H	e	l	l	o	\0
---	---	---	---	---	----

- Is comprised of characters between the ""

A character or a string constant?

- A **character constant** is a single character, enclosed in single quotes:

`'C'`

- A **string constant** is a sequence of characters enclosed in double quotes:

`"Hello, there!"`

- A single character in double quotes is a string constant, not a character constant:

`"C"`

The C++ `string` Class

- Must `#include <string>` to create and use string objects
- Can define `string` variables in programs
`string name;`
- Can assign values to string variables with the assignment operator
`name = "George";`
- Can display them with `cout`
`cout << name;`

Determining the Size of a Data Type

The **sizeof** operator gives the size of any data type or variable

```
double amount;
```

```
cout << "A float is stored in "  
      << sizeof(float) << " bytes\n";
```

```
cout << "Variable amount is stored in "  
      << sizeof(amount) << " bytes\n";
```

Variable Assignment vs. Initialization

- Initializing a variable
 - Gives an initial value to a variable at the time it is created
 - Can initialize some or all variables of definition

```
int length = 12;  
int width = 7, height = 5, area;
```

More on Variable Assignments and Initialization

- Assigning a value to a variable
 - Assigns a value to a previously created variable
 - A single variable name must appear on left side of the **=** symbol

```
int size;
```

```
size = 5; // legal
```

```
5 = size; // not legal
```


Scope

- The **scope** of a variable is that part of the program where the variable may be used
- A variable cannot be used before it is defined

```
int a;  
cin >> a;    // legal  
cin >> b;    // illegal  
int b;
```

cout <<

In-Class Exercise

- Trace the following program. Can it be compiled?

```
#include <iostream>
using namespace std;
```

```
int main()
{
    cout << value;

    int value;
    return 0;
}
```